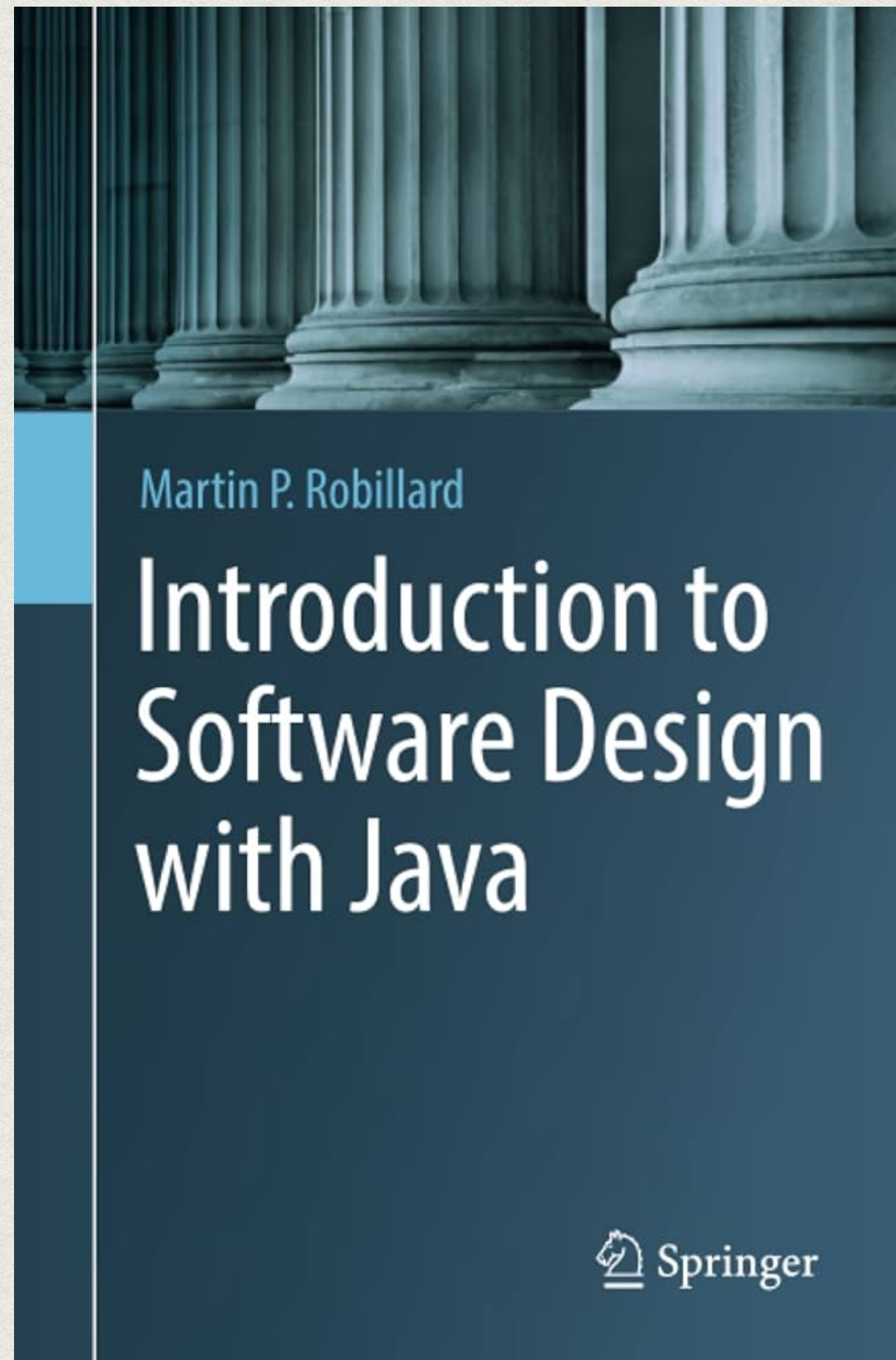




M1 MIAGE / MAPC ENCAPSULATION

Pascal POIZAT

RÉFÉRENCES



Conception Logicielle
Site web du cours INF5153 de l'UQAM

- [Retourner à la page d'accueil](#)

Introduction

Notes de cours

- [Introduction au cours](#)
- [Écosystème de développement](#)
- [Schotten Totten](#)

Capsules Vidéos

Introduction au cours

Introduction du cours de Conception Logicielle [IN...]

Partager

Génie logiciel : Conception

Regarder sur YouTube

Sébastien Mosser - INF5153
Chapitre 0 - Capsule 1
Automne 2020

UQAM - Département d'Informatique crédits photos: Pixabay

ENCAPSULATION

- **décomposition** en abstractions plus faciles à gérer
- limiter le nombre de points de contact (**couplage**)
- principe de **masquage** de l'information

EXEMPLE : CARTES

```
int card = 13; // As de coeur  
int suit = card / 13;  
int rank = card % 13;
```


EXEMPLE : CARTES

```
int card = 13; // As de coeur  
int suit = card / 13;  
int rank = card % 13;
```

```
int[] card = {1, 0}; // As de coeur
```


EXEMPLE : CARTES

- pas de correspondance au **métier**
- **couplage fort**
abstraction / implémentation
- modification **complexe**
facilité de **corruption** des données
- **PrimitiveObsession⁺**

```
int card = 13; // As de coeur  
int suit = card / 13;  
int rank = card % 13;
```

```
int[] card = {1, 0}; // As de coeur
```


EXEMPLE : CARTES

- pas de correspondance au **métier**
- **couplage fort**
abstraction / implémentation
- modification **complexe**
facilité de **corruption** des données
- **PrimitiveObsession+**

```
int card = 13; // As de coeur
int suit = card / 13;
int rank = card % 13;
```

```
int[] card = {1, 0}; // As de coeur
```

énumérations

```
public enum Suit { CLUBS, DIAMONDS,
                  SPADES, HEARTS }
```

```
public enum Rank { ACE, TWO, ...,
                  QUEEN, KING }
```

classe

```
public class Card {
    Suit aSuit;
    Rank aRank;
}
```


EXAMPLE : DECK

```
List<Card> deck = new ArrayList<>();
```


EXAMPLE : DECK

```
List<Card> deck = new ArrayList<>();
```

```
public class Deck {  
    List<Card> aCards = new ArrayList<>();  
}
```


EXAMPLE : DECK

```
List<Card> deck = new ArrayList<>();
```

```
public class Deck {  
    List<Card> aCards = new ArrayList<>();  
}
```

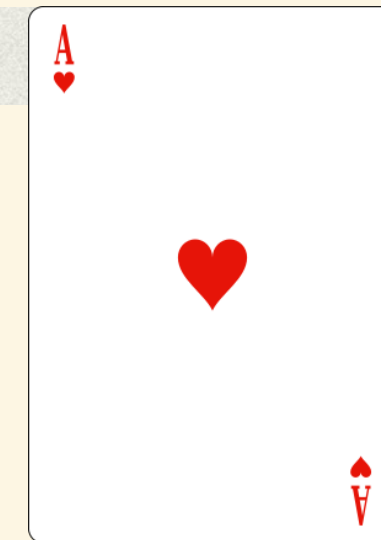
```
public class Card {  
    public Suit aSuit;  
    public Rank aRank;  
}  
public class Deck {  
    public List<Card> aCards  
        = new ArrayList<>();  
}
```


EXAMPLE : DECK

```
List<Card> deck = new ArrayList<>();
```

```
public class Deck {  
    List<Card> aCards = new ArrayList<>();  
}
```

```
public class Card {  
    public Suit aSuit;  
    public Rank aRank;  
}
```



```
public class Deck {  
    public List<Card> aCards  
        = new ArrayList<>();  
}
```

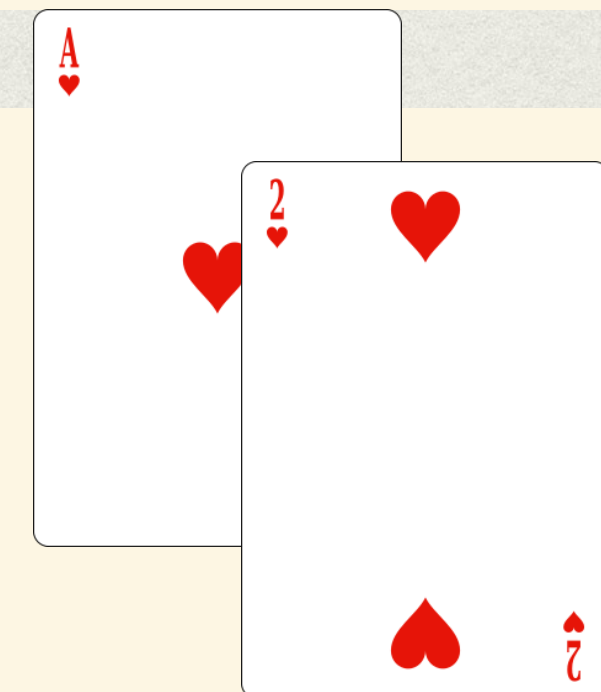

EXAMPLE : DECK

```
List<Card> deck = new ArrayList<>();
```

```
public class Deck {  
    List<Card> aCards = new ArrayList<>();  
}
```

```
public class Card {  
    public Suit aSuit;  
    public Rank aRank;  
}
```

```
public class Deck {  
    public List<Card> aCards  
        = new ArrayList<>();  
}
```



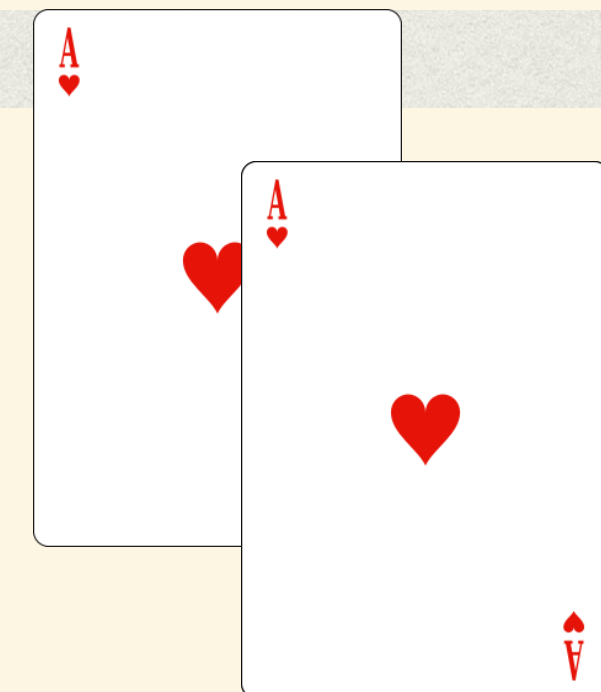
EXAMPLE : DECK

```
List<Card> deck = new ArrayList<>();
```

```
public class Deck {  
    List<Card> aCards = new ArrayList<>();  
}
```

```
public class Card {  
    public Suit aSuit;  
    public Rank aRank;  
}
```

```
public class Deck {  
    public List<Card> aCards  
        = new ArrayList<>();  
}
```



EXEMPLE : DECK

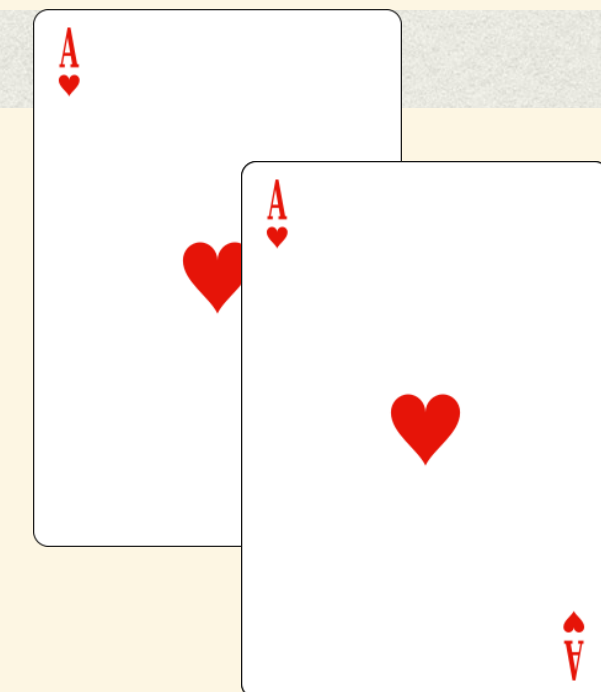
- encoder les concepts métier (abstractions / types)
n'est que la **première** étape !
- attention à l'utilisation de l'information hors de tout principes
- nécessité de **masquage** d'information
notion de **scopes**
visibilité : public/privé/protégé/pack.
- interface (méthodes publiques typiquement)
vs implémentation privée

```
List<Card> deck = new ArrayList<>();
```

```
public class Deck {  
    List<Card> aCards = new ArrayList<>();  
}
```

```
public class Card {  
    public Suit aSuit;  
    public Rank aRank;  
}
```

```
public class Deck {  
    public List<Card> aCards  
        = new ArrayList<>();  
}
```



EXAMPLE : DECK

```
public class Card {
    private Suit aSuit;
    private Rank aRank;
    public Card(Rank pRank, Suit pSuit) {
        aRank = pRank;
        aSuit = pSuit;
    }
    public Rank getRank() { return aRank; }
    public Suit getSuit() { return aSuit; }
}

public class Deck {
    public List<Card> aCards
        = new ArrayList<>();
}
```


EXEMPLE : DECK

- métier = classes / enums : OK
- encapsulation : OK
- masquage information : OK
- interaction contrôlée : OK

```
public class Card {  
    private Suit aSuit;  
    private Rank aRank;  
    public Card(Rank pRank, Suit pSuit) {  
        aRank = pRank;  
        aSuit = pSuit;  
    }  
    public Rank getRank() { return aRank; }  
    public Suit getSuit() { return aSuit; }  
}  
  
public class Deck {  
    public List<Card> aCards  
        = new ArrayList<>();  
}
```

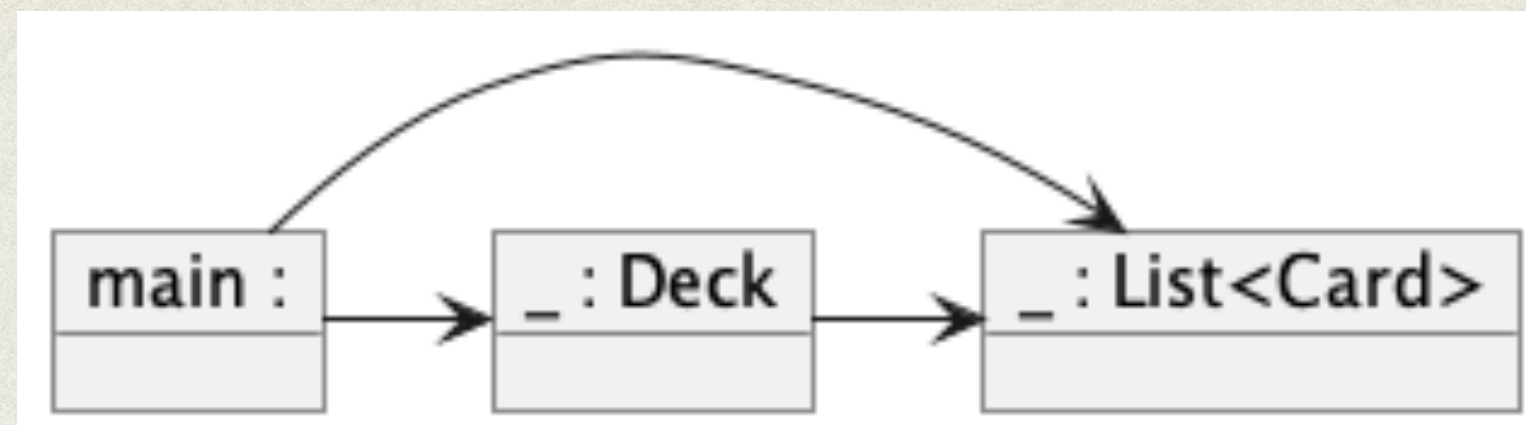

ECHAPPEMENT DE RÉFÉRENCES

```
public class Deck {  
    private List<Card> aCards = new ArrayList<>();  
    public Deck() { // ajouter 52 cartes + mélanger  
    }  
    public Card draw() { return aCards.remove(0); }  
}
```

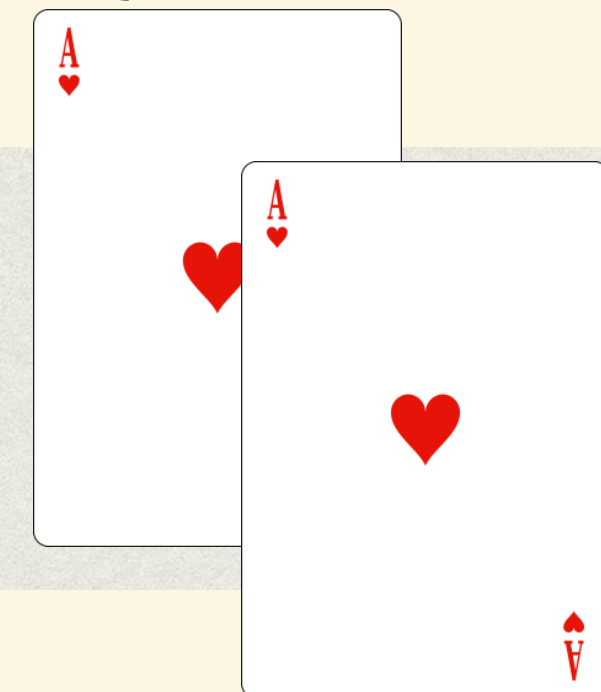

ECHAPPEMENT DE RÉFÉRENCES

```
public class Deck {  
    private List<Card> aCards = new ArrayList<>();  
    public Deck() { // ajouter 52 cartes + mélanger  
    }  
    public Card draw() { return aCards.remove(0); }  
    public List<Card> getCards() { return aCards; }  
}
```


ECHAPPEMENT DE RÉFÉRENCES

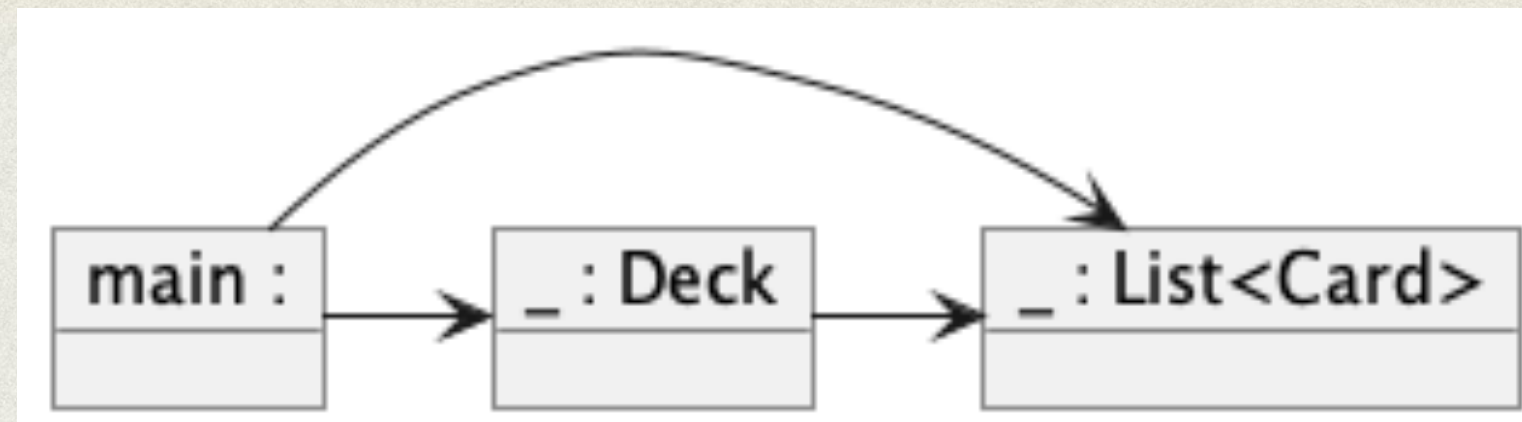


```
public class Deck {  
    private List<Card> aCards = new ArrayList<>();  
    public Deck() { // ajouter 52 cartes + mélanger  
    }  
    public Card draw() { return aCards.remove(0); }  
    public List<Card> getCards() { return aCards; }  
}
```



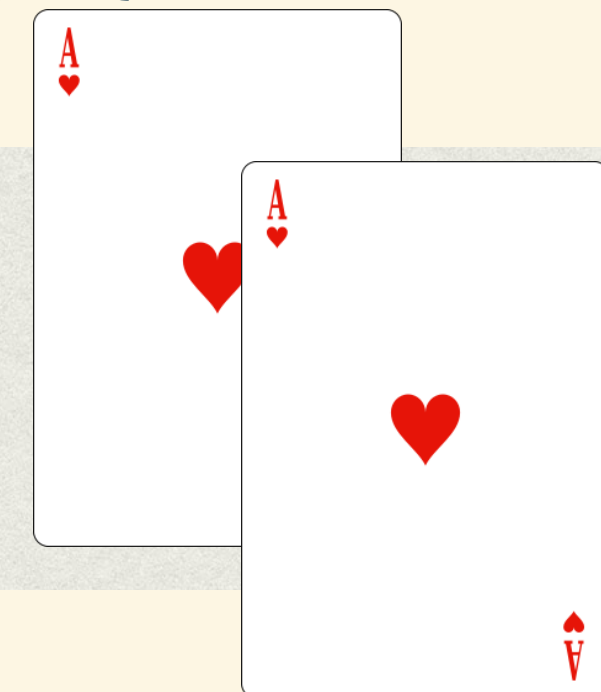
```
Deck deck = new Deck();  
List<Card> cards = deck.getCards();  
cards.add(new Card(Rank.ACE, Suit.HEARTS));
```


ECHAPPEMENT DE RÉFÉRENCES



- **échappement** de aCards
- on devrait passer par l'interface si on veut modifier les données

```
public class Deck {  
    private List<Card> aCards = new ArrayList<>();  
    public Deck() { // ajouter 52 cartes + mélanger  
    }  
    public Card draw() { return aCards.remove(0); }  
    public List<Card> getCards() { return aCards; }  
}
```



```
Deck deck = new Deck();  
List<Card> cards = deck.getCards();  
cards.add(new Card(Rank.ACE, Suit.HEARTS));
```


ECHAPPEMENT DE RÉFÉRENCES

- éviter les getters / setters **automatiques**
conception de mauvaise qualité
abstraction non effective
InappropriateIntimacy+
- même problème avec les **références externes**
utilisées lors de la **création** d'objets
(idem avec setter)
- et on peut avoir de nombreux niveaux à analyser

ECHAPPEMENT DE RÉFÉRENCES

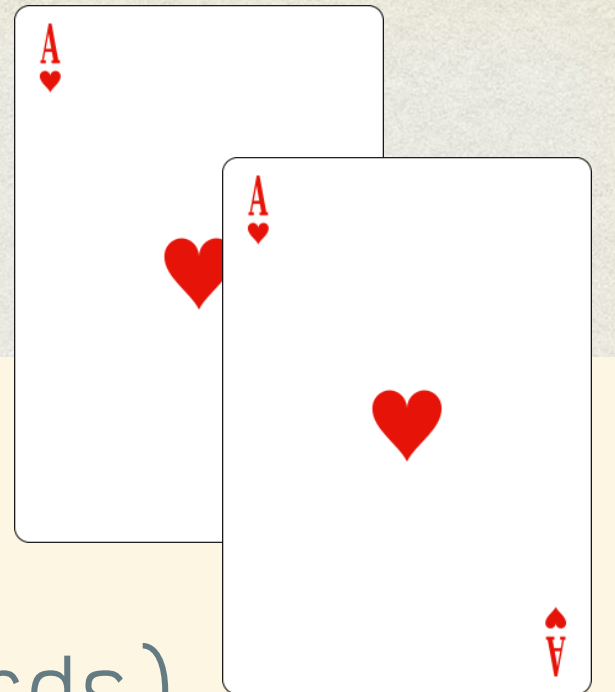
- éviter les getters / setters **automatiques**

conception de mauvaise qualité

abstraction non effective

InappropriateIntimacy+

```
public class Deck {  
    private List<Card> aCards;  
    public Deck(List<Card> pCards)  
        { aCards = pCards; }  
}
```



- même problème avec les **références externes**

utilisées lors de la **création** d'objets
(idem avec setter)

```
List<Card> cards = new ArrayList<>();  
Deck deck = new Deck(cards);  
cards.add(new Card(Rank.ACE, Suit.HEARTS));
```

- et on peut avoir de nombreux niveaux à analyser

IMMUTABILITÉ

- on veut éviter de modifier les données sans passer par l'interface
- éviter la fuite de données ... sauf si **immuables**
- en pratique l'immuabilité est souvent **souhaitable**
- **attention** en Java
String, (enums), (final)

```
public class Card {  
    private Suit aSuit;  
    private Rank aRank;  
    public Card(Rank pRank, Suit pSuit) {  
        aRank = pRank;  
        aSuit = pSuit;  
    }  
    public Rank getRank() { return aRank; }  
    public Suit getSuit() { return aSuit; }  
}
```


EXPOSER DES DONNÉES

- pas l'implémentation !
- **différentes** solution
(avec + et -)
- **extension** de l'interface
- retour de **copies**
ou **wrappers** d'immuabilité,
voire copies profondes

```
public class Deck {  
    private List<Card> aCards = new ArrayList<>();  
    public Deck() { ... }  
    public Card draw() { return aCards.remove(0); }  
    public int size() { return aCards.size(); }  
    public Card getCard(int pIndex)  
        { return aCards.get(pIndex); }  
}
```

```
public List<Card> getCards() {  
    return new ArrayList<>(aCards);  
    // ou :  
    // return Collections.unmodifiableList(aCards);  
}
```


CONTRATS

```
Card boum = new Card(null, Suit.HEARTS);
```


CONTRATS

- un **contrat** peut parfois être intégré à l'interface (typage)
- souvent besoin d'aller **au delà** de la simple interface
- programmation / conception par contrats
pré-, post-, invariant (+ autres)
- annotations « maison », assert, JML, OCL

```
Card boum = new Card(null, Suit.HEARTS);
```


CONTRATS

- un **contrat** peut parfois être intégré à l'interface (typage)
- souvent besoin d'aller **au delà** de la simple interface
- programmation / conception par contrats pré-, post-, invariant (+ autres)
- annotations « maison », assert, JML, OCL

```
Card boum = new Card(null, Suit.HEARTS);
```

Java doc

```
/**
 * Constructor for {@link Card}s.
 * @param pSuit the {@link Suit} of ...
 * @param pRank the {@link Rank} of ...
 * @pre pSuit != null && pRank != null
 */
public Card(Suit pSuit, Rank pRank) {
    assert pSuit != null;
    assert pRank != null;
    aSuit = pSuit;
    aRank = pRank;
}
```

assertions

CONTRATS

- un **contrat** peut parfois être intégré à l'interface (typage)
- souvent besoin d'aller **au delà** de la simple interface
- programmation / conception par contrats pré-, post-, invariant (+ autres)
- annotations « maison », assert, JML, OCL

```
Card boum = new Card(null, Suit.HEARTS);
```

Java doc

```
/**
 * Constructeur pour les cartes.
 * @param pSuit le Suit de la carte
 * @param pRank le Rank de la carte
 * @pre pSuit != null && pRank != null
 */
public Card(Suit pSuit, Rank pRank) {
    assert pSuit != null;
    assert pRank != null;
    aSuit = pSuit;
    aRank = pRank;
}
```

assertions

SYNTHÈSE

- concepts métier : **classes**
éviter **PrimitiveObsession⁺**
- collection d'un petit nombre d'éléments :
énumération
- **cacher** l'implémentation interne
attention aux fuites de références
éviter **InappropriateIntimacy⁺**
- exposer les données
c'est différent d'exposer l'implémentation
- **immutabilité** si possible
- **contrats** si possible

